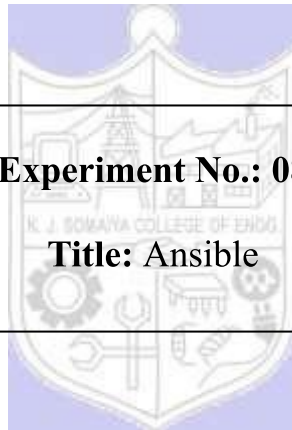


Experiment No.: 08

Title: Ansible



Batch: B-1**Roll No.: 16010422234****Experiment No.: 08**

Aim: To write an Ansible playbook.

Resources needed: Killercodea (Online)

Theory:

Ansible provides open-source automation that reduces complexity and runs everywhere. Using Ansible lets you automate virtually any task. Here are some common use cases for Ansible:

- Eliminate repetition and simplify workflows
- Manage and maintain system configuration
- Continuously deploy complex software
- Perform zero-downtime rolling updates

Ansible uses simple, human-readable scripts called playbooks to automate your tasks. You declare the desired state of a local or remote system in your playbook. Ansible ensures that the system remains in that state.

Ansible is designed around the following principles:

Agent-less architecture

Low maintenance overhead by avoiding the installation of additional software across IT infrastructure.

Simplicity

Automation playbooks use straightforward YAML syntax for code that reads like documentation. Ansible is also decentralized, using SSH existing OS credentials to access to remote machines.

Scalability and flexibility

Easily and quickly scale the systems you automate through a modular design that supports a large range of operating systems, cloud platforms, and network devices.

Idempotence and predictability

When the system is in the state your playbook describes, Ansible does not change anything, even if the playbook runs multiple times.

Ansible Playbooks

Ansible Playbooks offer a repeatable, re-usable, simple configuration management and multi-machine deployment system, one that is well suited to deploying complex applications. If you need to execute a task with Ansible more than once, write a playbook and put it under source control. Then you can use the playbook to push out new configuration or confirm the configuration of remote systems. The playbooks in the Ansible-examples repository illustrate many useful techniques. You may want to look at these in another tab as you read the documentation.

Procedure:

1. Install Ansible and create node-1, node-2, node-3 server. As shown in figure.
<https://killercoda.com/Ansible/scenario/Ansible-en-101>
2. Create your own playbook.

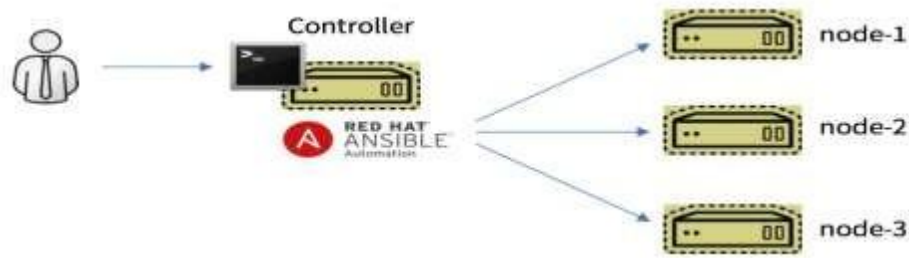


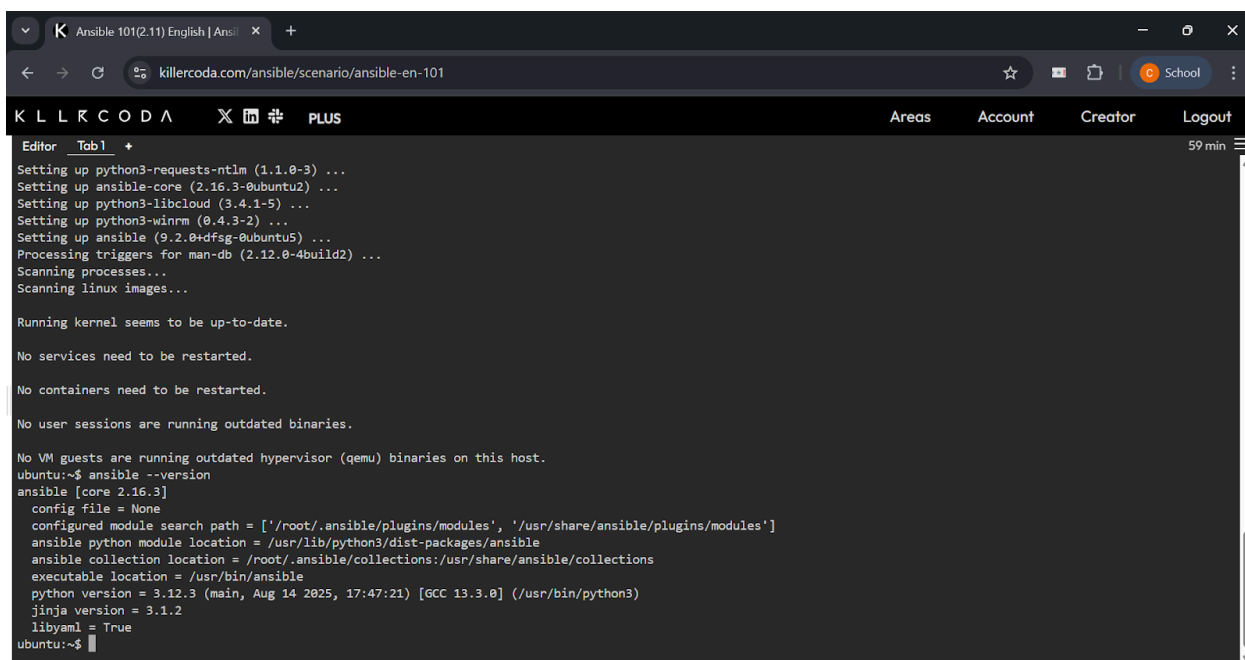
Figure 1 Ansible Automation

Results: (Document with screenshots)

1. Steps with screenshots

```

K Ansible 101(2.11) English | Ansil x +
killercoda.com/ansible/scenario/ansible-en-101
K L L K C O D A X # PLUS Areas Account Creator Logout 60 min
Editor Tab 1 +
ubuntu:~$ apt install ansible-core
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following package was automatically installed and is no longer required:
  squashfs-tools
Use 'apt autoremove' to remove it.
The following additional packages will be installed:
  ansible python3-argcomplete python3-dnspython python3-kerberos python3-libcloud python3-lockfile python3-ntlm-auth
  python3-passlib python3-requests-ntlm python3-resolvelib python3-selinux python3-simplejson python3-winnm
  python3-xmltodict
Suggested packages:
  cowsay sshpass python3-trio python3-aiquic python3-h2 python3-httpx python3-httpcore python-lockfile-doc
The following NEW packages will be installed:
  ansible ansible-core python3-argcomplete python3-dnspython python3-kerberos python3-libcloud python3-lockfile
  python3-ntlm-auth python3-passlib python3-requests-ntlm python3-resolvelib python3-selinux python3-simplejson
  python3-winnm python3-xmltodict
0 upgraded, 15 newly installed, 0 to remove and 41 not upgraded.
Need to get 19.5 MB of archives.
After this operation, 315 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://archive.ubuntu.com/ubuntu noble/universe amd64 python3-resolvelib all 1.0.1-1 [25.7 kB]
Get:2 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-dnspython all 2.6.1-1ubuntu1 [163 kB]
Get:3 http://archive.ubuntu.com/ubuntu noble/universe amd64 ansible-core all 2.16.3-0ubuntu2 [1280 kB]
Get:4 http://archive.ubuntu.com/ubuntu noble/universe amd64 ansible all 9.2.0+dfsg-0ubuntu5 [16.4 MB]
Get:5 http://archive.ubuntu.com/ubuntu noble-updates/universe amd64 python3-argcomplete all 3.1.4-1ubuntu0.1 [33.8 kB]
Get:6 http://archive.ubuntu.com/ubuntu noble/universe amd64 python3-kerberos amd64 1.1.14-3.1build9 [21.2 kB]
Get:7 http://archive.ubuntu.com/ubuntu noble/universe amd64 python3-lockfile all 1:0.12.2-3 [13.7 kB]
Get:8 http://archive.ubuntu.com/ubuntu noble/main amd64 python3-simplejson amd64 3.19.2-1build2 [54.5 kB]
  
```



```
Setting up python3-requests-ntlm (1.1.0-3) ...
Setting up ansible-core (2.16.3-0ubuntu2) ...
Setting up python3-libcloud (3.4.1-5) ...
Setting up python3-winrm (0.4.3-2) ...
Setting up ansible (9.2.0+dfsg-0ubuntu5) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

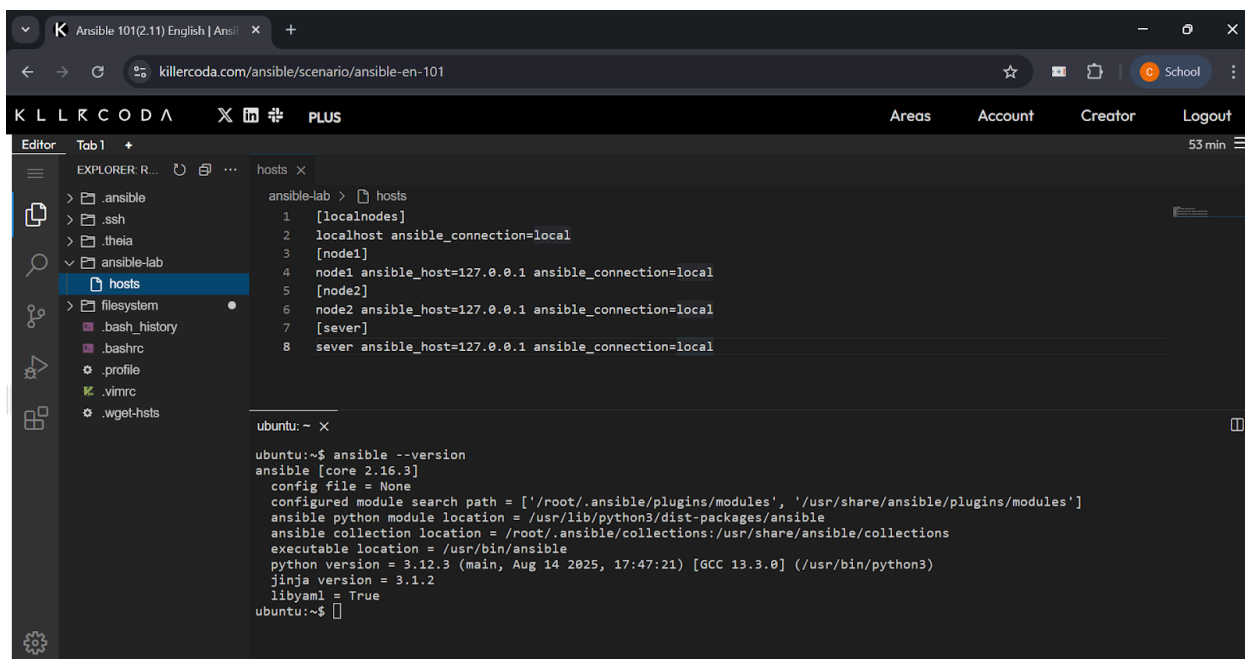
Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

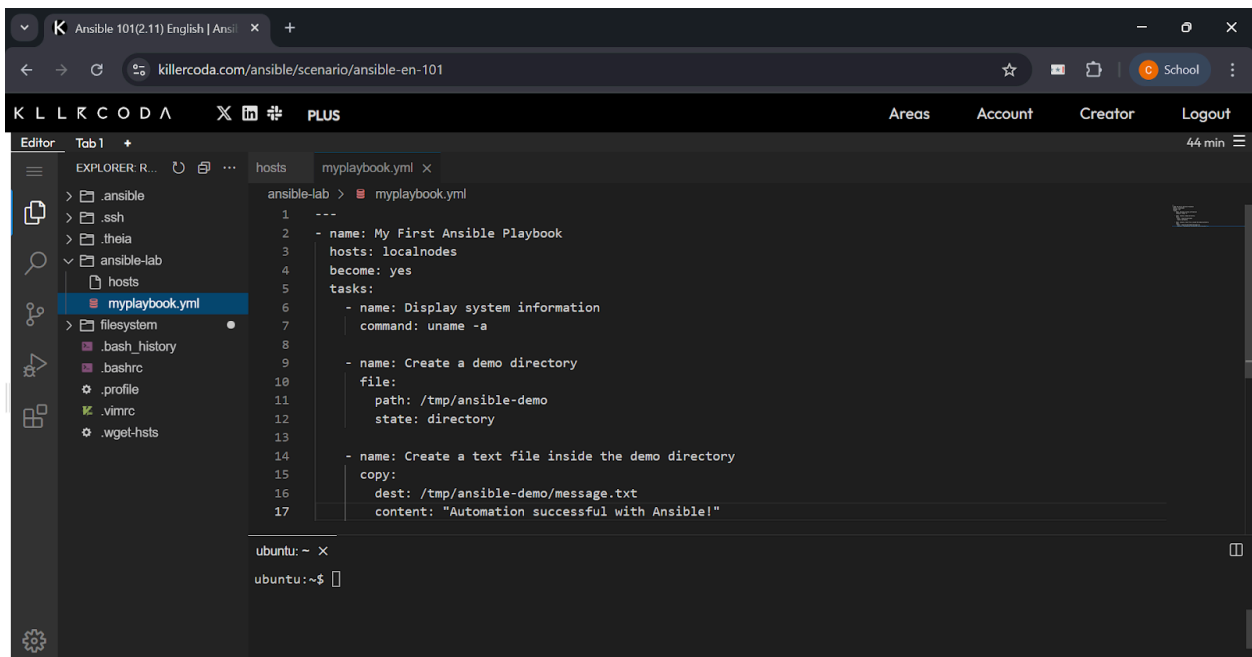
No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu:~$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
ubuntu:~$
```



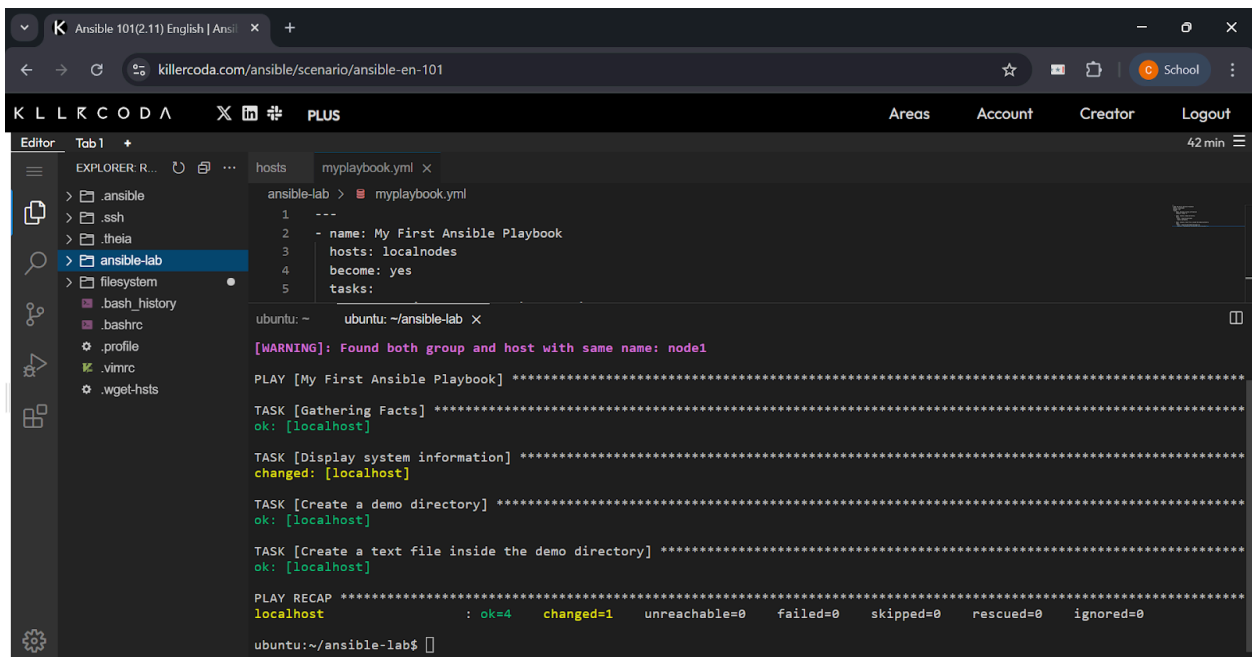
```
ansible-lab > hosts
1 [localnodes]
2 localhost ansible_connection=local
3 [node1]
4 node1 ansible_host=127.0.0.1 ansible_connection=local
5 [node2]
6 node2 ansible_host=127.0.0.1 ansible_connection=local
7 [sever]
8 sever ansible_host=127.0.0.1 ansible_connection=local

ubuntu:~$
ubuntu:~$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/root/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /root/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
ubuntu:~$
```



```
1 ---
2 - name: My First Ansible Playbook
3   hosts: localnodes
4   become: yes
5   tasks:
6     - name: Display system information
7       command: uname -a
8
9     - name: Create a demo directory
10      file:
11        path: /tmp/ansible-demo
12        state: directory
13
14     - name: Create a text file inside the demo directory
15       copy:
16         dest: /tmp/ansible-demo/message.txt
17         content: "Automation successful with Ansible!"
```

ubuntu: ~ X
ubuntu: ~\$



```
ubuntu: ~ X
[WARNING]: Found both group and host with same name: node1

PLAY [My First Ansible Playbook] *****

TASK [Gathering Facts] *****
ok: [localhost]

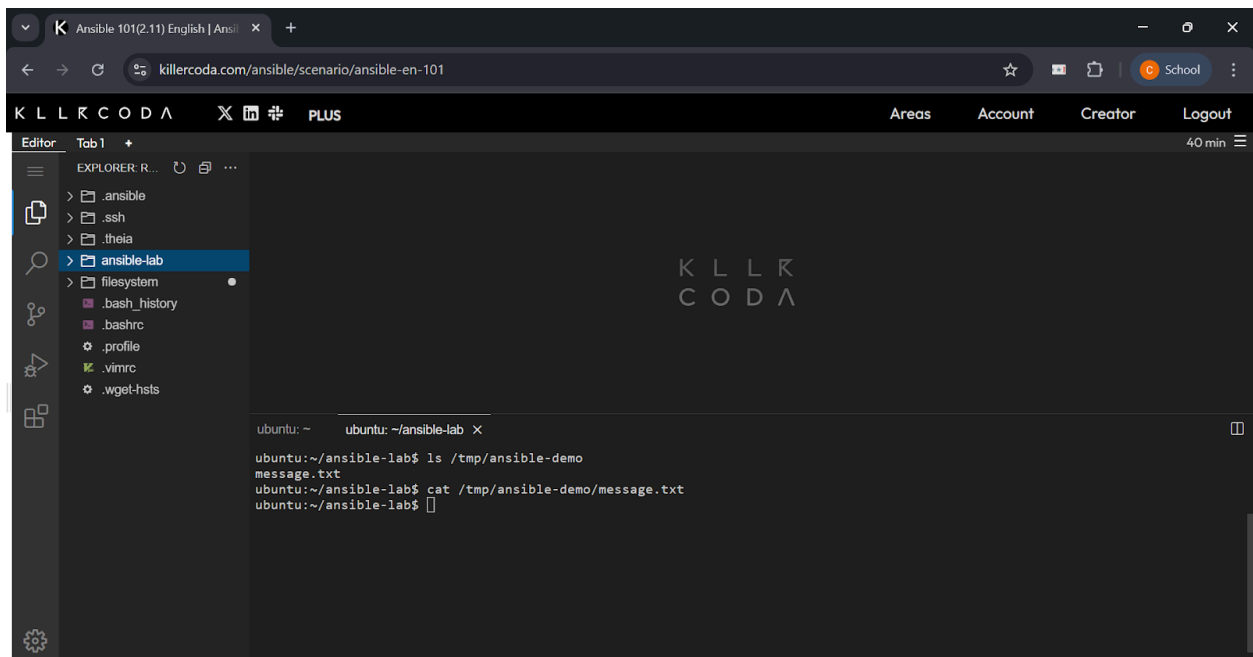
TASK [Display system information] *****
changed: [localhost]

TASK [Create a demo directory] *****
ok: [localhost]

TASK [Create a text file inside the demo directory] *****
ok: [localhost]

PLAY RECAP *****
localhost : ok=4  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

ubuntu: ~/ansible-lab$
```



Questions:

1. Difference between Ansible vs Puppet vs Chef.

Ans: Ansible, Puppet, and Chef are all popular open-source configuration management tools used for automating IT infrastructure. Here's a comparison:

Ansible:

- Architecture: Agent-less (uses SSH for communication).
- Configuration Language: YAML (Playbooks) - known for its simplicity and human-readability.
- Ease of Use: Generally considered the easiest to learn and set up due to its agent-less nature and simple YAML syntax.
- Scalability: Highly scalable, with Red Hat Ansible Automation Platform providing enterprise features for large environments.
- Management Model: Primarily push-based, where the control machine pushes configurations to the managed nodes.
- Written In: Python.

Puppet:

- Architecture: Agent-based (requires a Puppet agent on each managed node that communicates with a Puppet Master).
- Configuration Language: Puppet DSL (Domain-Specific Language) based on Ruby.
- Ease of Use: Relatively easier to learn than Chef but has a steeper learning curve than Ansible, requiring understanding of its DSL.
- Scalability: Designed for large-scale infrastructure management with a robust master-client architecture.

- Management Model: Primarily pull-based, where agents on the managed nodes pull configurations from the Puppet Master at regular intervals.
- Written In: Ruby.

Chef:

- Architecture: Agent-based (requires a Chef client on each node, communicating with a Chef server, with a workstation for development).
- Configuration Language: Ruby DSL (Cookbooks and Recipes) - requires programming skills in Ruby.
- Ease of Use: Has the steepest learning curve among the three due to its reliance on Ruby programming.
- Scalability: Highly scalable, transforming infrastructure into code for dynamic environments.
- Management Model: Primarily pull-based, with clients pulling configurations (cookbooks and recipes) from the Chef server.
- Written In: Ruby.

In summary, Ansible prioritizes simplicity and an agent-less approach, making it quick to deploy and easy to learn. Puppet and Chef are agent-based, offering more granular control and state management, but with higher complexity and a steeper learning curve, particularly Chef with its Ruby-centric approach.

2. Explain agent less and agent base with examples.

Ans: The difference between agent-less and agent-based architectures lies in how the management system communicates with and controls the target machines.

Agent-less Architecture

- Definition: In an agent-less system, there is no specialized software or "agent" installed on the target machines (nodes) that are being managed. Instead, the central management server (control node) uses standard, pre-existing protocols to connect to and manage the remote systems.
- How it works: For Linux/Unix machines, Ansible typically uses SSH (Secure Shell) to connect, execute commands, and transfer files. For Windows machines, it might use WinRM (Windows Remote Management). These protocols are usually already present on the systems, eliminating the need for additional installations.
- Advantages:
 - Easier setup and maintenance: No agents to install, update, or troubleshoot on each managed node.
 - Reduced overhead: No extra processes running on the target systems, thus minimal impact on their performance.
 - Lower security risk: Fewer components to secure and fewer potential attack vectors.
- Disadvantages:

- Dependency on existing protocols: Relies on SSH or WinRM being correctly configured and accessible.
- Less real-time visibility: May not provide as granular or real-time monitoring data as agent-based systems, as it typically operates by pushing commands or pulling information at intervals.
- Example: Ansible is a prime example. An Ansible control node connects to target servers via SSH, executes tasks defined in playbooks, and ensures the desired state. There's no Ansible software daemon running permanently on the managed servers.

Agent-based Architecture

- Definition: In an agent-based system, a small piece of software, an "agent," is installed and runs continuously on each target machine that needs to be managed. This agent is responsible for collecting data, executing commands, and communicating with a central management server.
- How it works: The agent on each node periodically checks in with the central management server (e.g., Puppet Master, Chef Server) to receive configuration instructions or policies. It then applies these configurations to its host system and reports its status back to the server.
- Advantages:
 - Real-time monitoring and control: Agents can provide continuous, granular data and respond immediately to changes.
 - Offline operation: Some agents can store policies and continue to enforce them even if they temporarily lose connection to the central server.
 - Deeper insights: Can collect more detailed system-level information.
- Disadvantages:
 - Complex setup and maintenance: Requires installation, configuration, and ongoing updates of agents on every managed machine.
 - Resource consumption: Agents consume CPU, memory, and network resources on the target systems.
 - Potential security vulnerabilities: Each agent represents an additional attack surface.
- Examples: Puppet and Chef are classic examples. In Puppet, a "Puppet agent" runs on each client machine and communicates with the "Puppet Master." Similarly, in Chef, a "Chef client" runs on each node and pulls configuration instructions from the "Chef server" and Chef "cookbooks."

Outcomes: CO4 – Explain code deployment and monitoring systems and their tool support.

Conclusion: (Conclusion to be based on the Results and outcomes achieved)

This experiment aims to demonstrate the practical application of Ansible playbooks for automated IT infrastructure management. Ansible, with its agent-less architecture and simple YAML syntax, provides a highly scalable and flexible solution for configuration management

and multi-machine deployment. Its idempotent nature ensures consistent system states, making it a powerful tool for deploying complex applications and maintaining system configurations efficiently. By automating tasks, Ansible significantly reduces manual overhead and promotes predictable IT operations.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

References:

Websites:

1. Ansible Hello World: <https://www.youtube.com/watch?v=vyiJrTHXE0M>
 2. Use Ansible Playbooks:
https://docs.Ansible.com/Ansible/latest/playbook_guide/index.html
-